

Data Structures



Lecture 10

Stack Application, Implementation, Queue Basics & Implementation

Prantik Paul [PNP]

Lecturer

Department of Computer Science and Engineering

BRAC University

Stack Applications (Parenthesis Matching)

(a (b + c) + d)

[(a b) (c d)]

([a { x y } b])

Correct

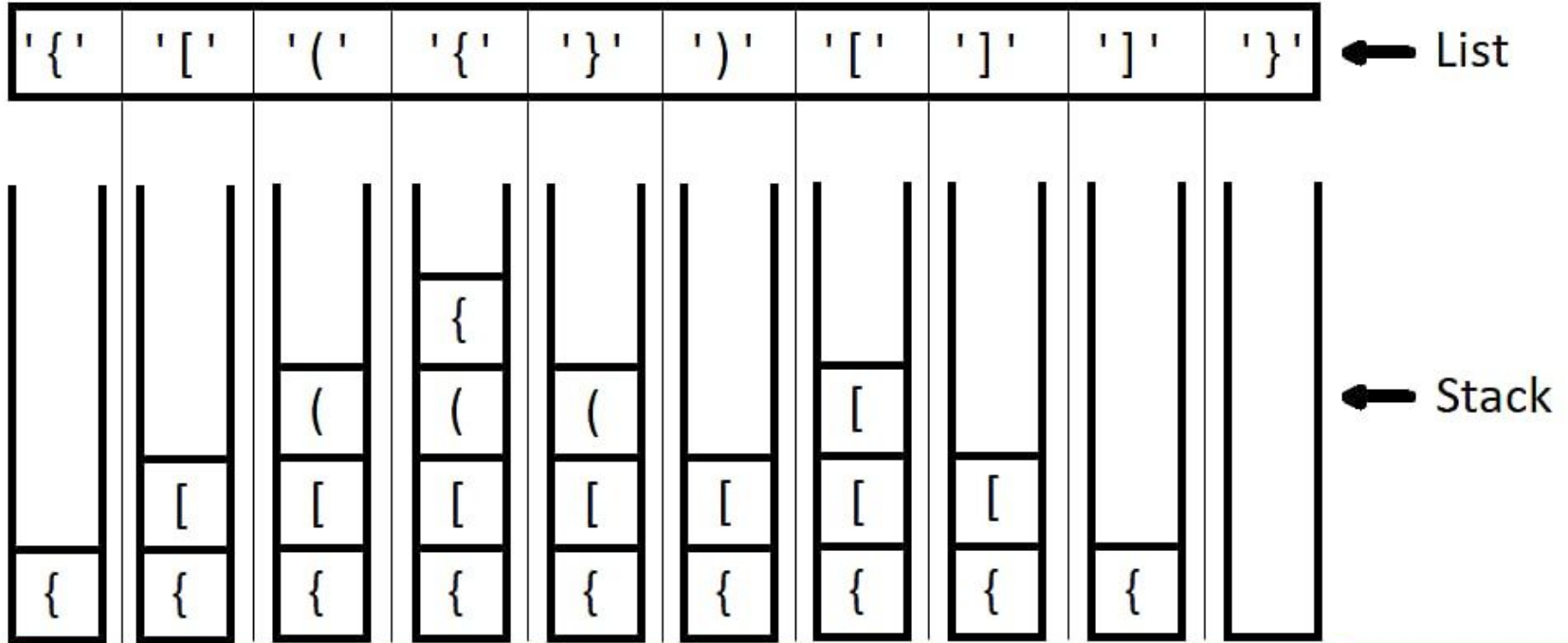
(a (b + c) + d

[(a b] (c d))

([a { x y) b])

Incorrect

Stack Applications (Parenthesis Matching)



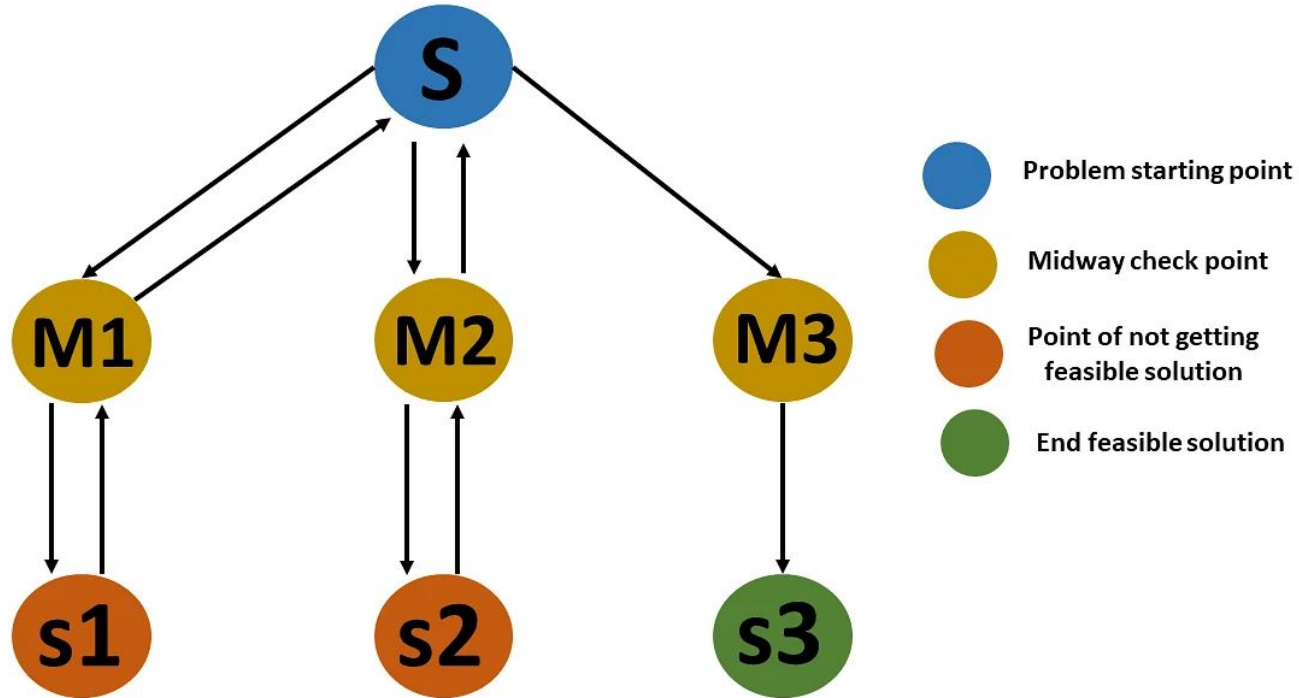
Stack Applications (Parenthesis Matching)

```
begin with an empty stack and an input stream (for the expression).
while there is more input to read, do:
    read the next input character;
    if it's an opening parenthesis/brace/bracket "(" or "{" or "["
        then push it onto the stack;
    if it's a closing parenthesis/brace/bracket ")" or "}" or "]"
        then pop the opening symbol from stack;
        compare the closing with opening symbol;
        if it matches
            then continue with next input character;
        if it does not match
            then return false;
end while;
// all matched, so return true
return true;
```

Stack Applications (BackTracking - Maze)



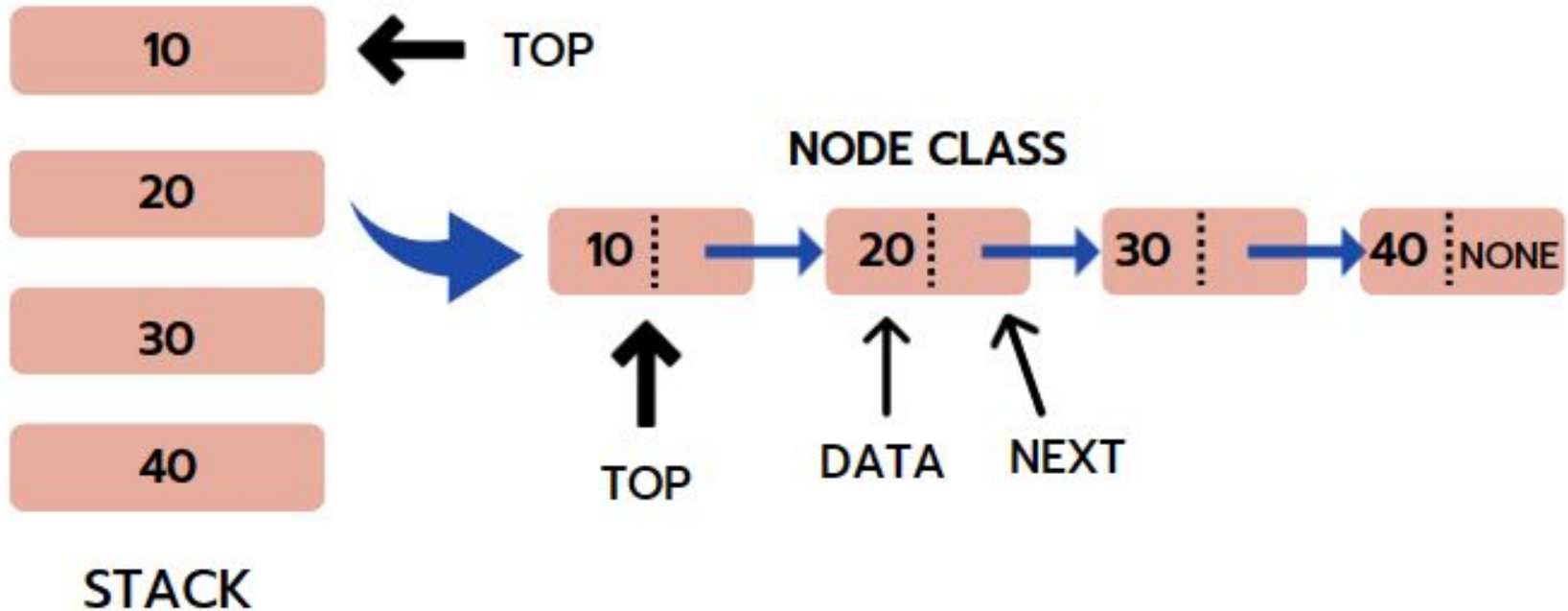
Stack Applications (BackTracking)



Stack Application

Reverse a Number using Stack

Stack Implementation (with Linked List)



Stack Implementation (Node Class)

```
class Node:  
    def __init__(self, element, next):  
        self.element = element  
        self.next = next
```

Stack Implementation (Push Operation)

```
class Stack:
    def __init__(self):
        self.top = None

    def push(self, elem):
        if self.top == None: #Stack is empty
            self.top = Node(elem, None)
        else:
            newNode = Node(elem, None)
            newNode.next = self.top
            self.top = newNode
```

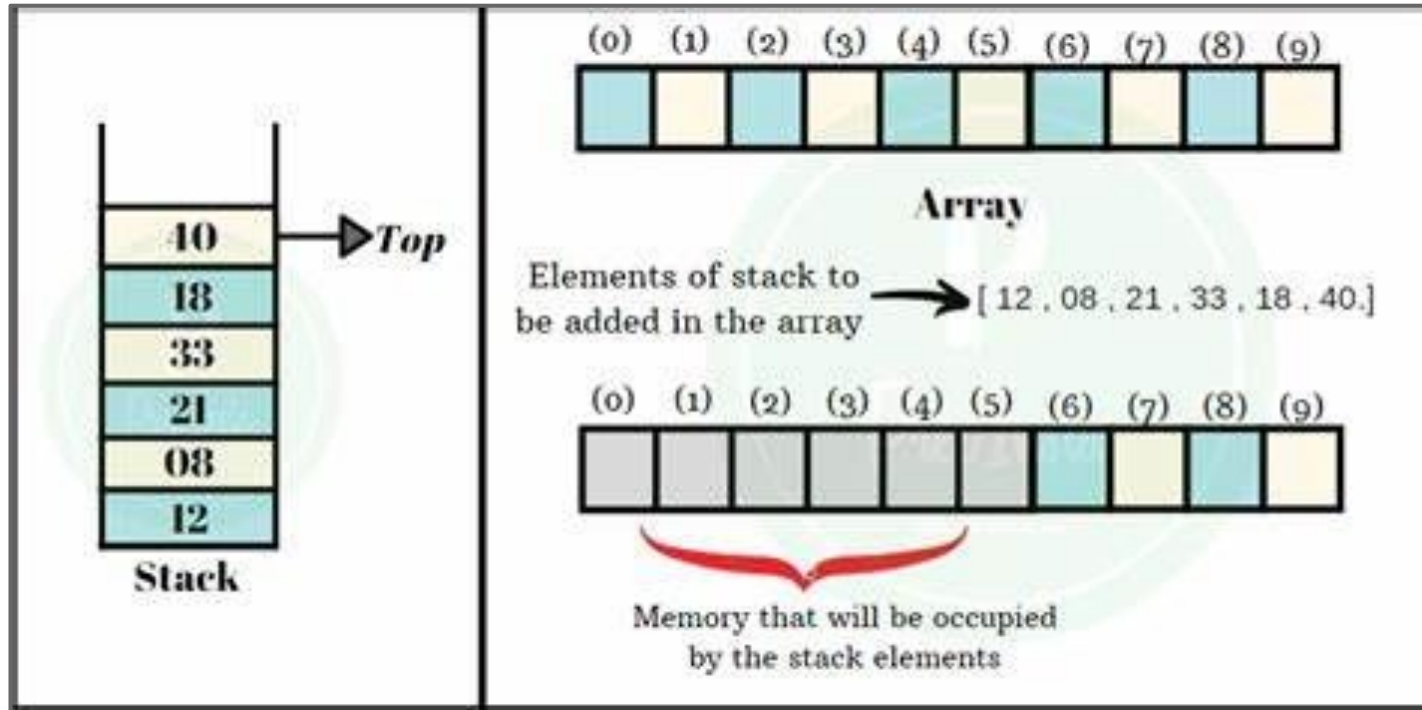
Stack Implementation (Pop Operation)

```
def pop(self):  
    if self.top == None:  
        return None #Stack Underflow  
    else:  
        popped = self.top  
        self.top = self.top.next  
        popped.next = None  
        return popped.element
```

Stack Implementation (Peek Operation)

```
def peek(self):  
    if self.top == None:  
        return None #Stack Underflow  
    else:  
        return self.top.element
```

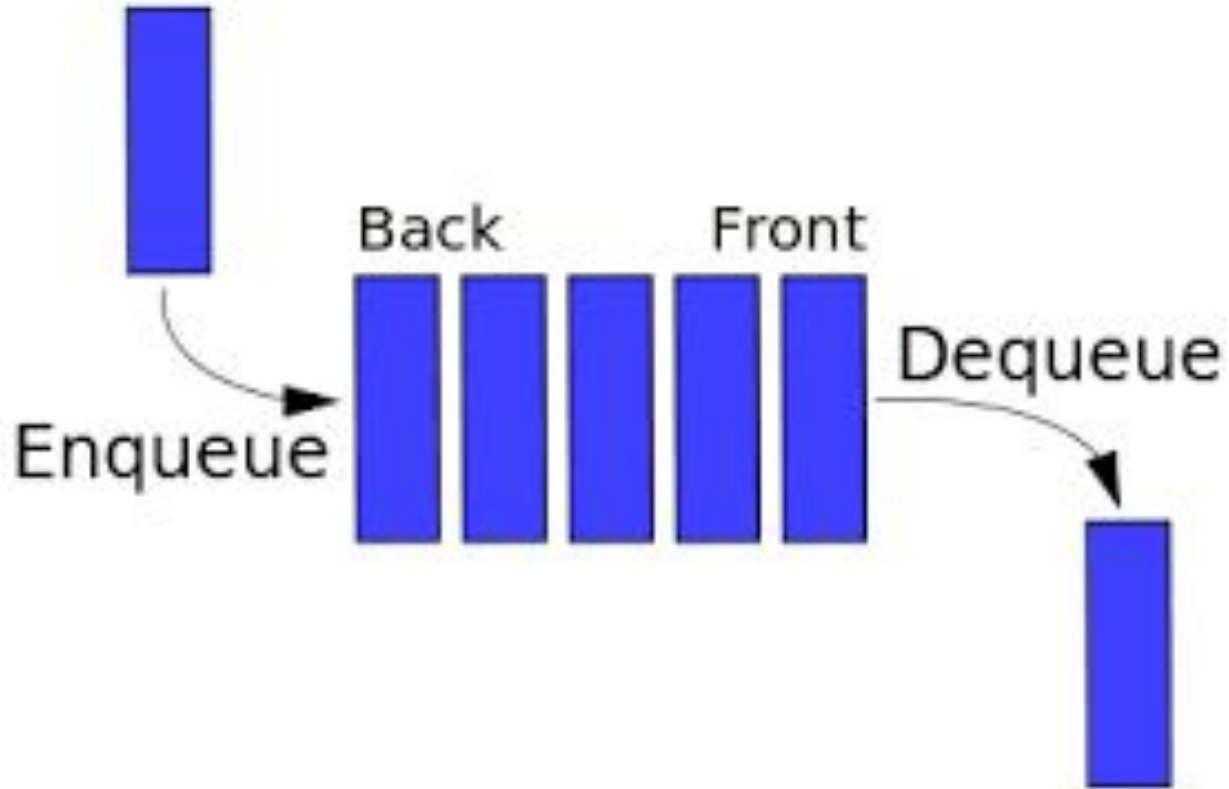
Stack Implementation (with Array)



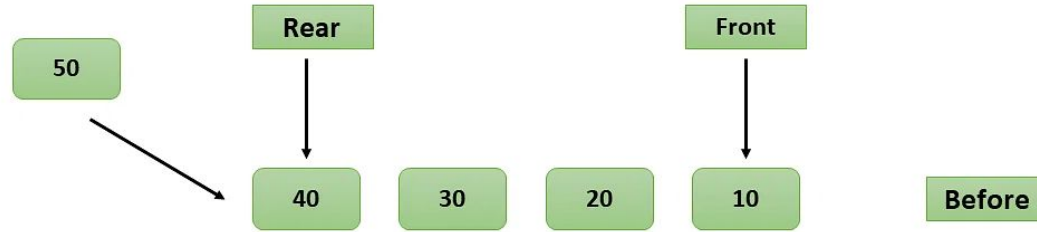
Queue (in real life)



Queue (Data Structure)



Queue (Enqueue)



Queue Enqueue

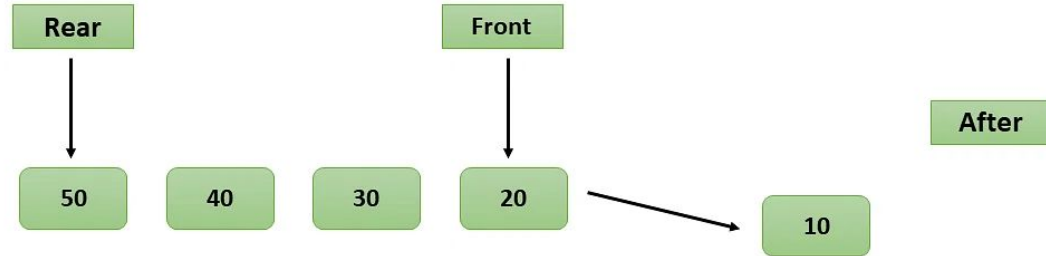
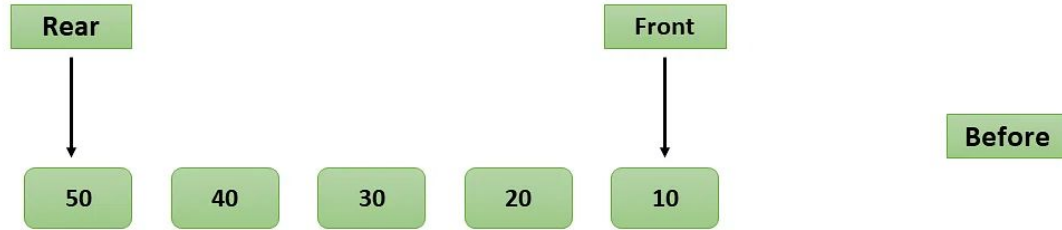
Queue (Enqueue)

```
class Queue:

    def __init__(self):
        self.front = None
        self.back = None

    def enqueue(self, elem):
        if self.front == None: # This means first item
            self.front = Node(elem, None)
            self.back = self.front # Initially front and back are same
        else:
            n = Node(elem, None)
            self.back.next = n # Inserting at the last
            self.back = self.back.next # Moving back at the end
```

Queue (Dequeue)



Queue Dequeue

Queue (Dequeue)

```
def dequeue(self):  
    if self.front == None: # For empty Queue  
        print("Queue Underflow Exception")  
    else:  
        dequeued_item = self.front.elem  
        self.front = self.front.next # Moving the front pointer  
        return dequeued_item # Returning the dequeued item
```

Queue (Peek)

```
def peek(self):  
    if self.front == None: # For empty Queue  
        print("Queue Underflow Exception")  
    else:  
        return self.front.elem # Returning the front item
```